

1. Introduction to Computer Vision

Computer vision enables computers to interpret, analyze, and understand visual information from the world (images or videos) to make decisions. Tasks include image processing, feature extraction, object detection, and segmentation, utilizing techniques from machine learning and AI. Applications range from autonomous vehicles (recognizing pedestrians/signs) and medical imaging (tumor detection) to facial recognition and augmented reality.

2. Basic Terminology

- **Image:** A 2D array of pixels, the smallest unit carrying color/intensity information.
- **Resolution:** Dimensions in pixels (e.g., 1920x1080).
- **Grayscale:** Pixels representing intensity only (0 black, 255 white).
- **RGB:** Red, Green, Blue color representation.
- **Feature Extraction:** Isolating relevant features from an image using filters.
- **Edge Detection:** Identifying boundaries/transitions.
- **Object Detection:** Localizing specific objects within a frame.
- **Segmentation:** Dividing an image into meaningful segments.

3. Essential CV Libraries

- **OpenCV:** General CV tasks, high abstraction, strong community.
- **Scikit-Image:** Python-independent image processing.
- **Pillow:** Basic image processing.
- **TorchVision:** Native integration with PyTorch.
- **TensorFlow / Keras:** High-level neural networks and general ML.
- **OpenVINO:** CV inference optimization (Intel).

4. Digital Image and Coordinate System

A digital image is a two-dimensional discrete function $f(x, y)$, where amplitude is the pixel intensity. The origin (0,0) is at the top-left corner, with the x-axis mapping to rows and the y-axis mapping to columns.

5. Fundamental Steps of Digital Image Processing (DIP)

1. **Image Acquisition:** Acquiring the image digitally.
2. **Image Enhancement:** Highlighting features like brightness and contrast.
3. **Image Restoration:** Improving visual appearance.
4. **Color Image Processing:** Color modeling and processing.
5. **Wavelets:** Multi-resolution processing and image division for compression.
6. **Compression:** Reducing storage requirements.
7. **Morphological Processing:** Extracting components for shape description.
8. **Segmentation:** Partitioning an image into distinct objects.
9. **Representation & Description:** Transforming raw pixel data to extract class-differentiating information.

- 10. **Object Recognition:** Assigning labels based on descriptors.
- 11. **Knowledge Base:** Using complex databases to locate important information.

6. Basic Image Operations

Reading and Displaying an Image

OpenCV uses BGR by default, so it must be converted to RGB for Matplotlib.

```
Python
import cv2
import matplotlib.pyplot as plt

image_path = 'Sukuna.jpeg'
image = cv2.imread(image_path)

if image is None:
    print("Error: Image not found.")
else:
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(8,6))
    plt.axis('off')
    plt.title('King of Curses - Ryomen Sukuna', fontsize=16, color='darkred', pad=15)
    plt.imshow(image_rgb)
    plt.show()
```

Grayscale Conversion

Requires `cmap='gray'` in Matplotlib to display correctly without false colors.

```
Python
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.imshow(gray_image, cmap='gray')
```

Resize Image

Scales the image proportionally.

```
Python
scale_factor = 0.5
resized_image = cv2.resize(image_rgb, None, fx=scale_factor, fy=scale_factor)
new_height, new_width = resized_image.shape[:2]
```

Image Filtering (Gaussian Blur)

Requires an odd-numbered kernel size (e.g., 31, 31).

Python

```
blurred_image = cv2.GaussianBlur(image_rgb, (31, 31), 0)
```

Cropping

Uses standard NumPy slicing.

Python

```
mid_x = width // 2  
cropped_image = image_rgb[:, :mid_x] # Takes left half
```

Adding Text

Draws text directly on the image.

Python

```
cv2.putText(image_rgb, "Domain Expansion", (150, 150), cv2.FONT_HERSHEY_SIMPLEX, 1.5,  
(255, 0, 0), 2)
```

Image Analysis with Pandas

Flattens the 3D array into a 2D array and analyzes the channels.

Python

```
import pandas as pd  
flattened_image = image.reshape(-1, 3)  
image_df = pd.DataFrame(flattened_image, columns=['B', 'G', 'R'])  
print(image_df.describe())
```

Image Thresholding

Forces pixels to pure black or pure white based on a threshold limit.

Python

```
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  
ret, thresholded_image = cv2.threshold(gray_image, 100, 200, cv2.THRESH_BINARY)
```

Image Rotation

Uses an affine matrix built from a center point, angle, and scale.

Python

```
center = (width / 2, height / 2)  
rotation_matrix = cv2.getRotationMatrix2D(center, 45, 1)  
rotated_image = cv2.warpAffine(image_rgb, rotation_matrix, (width, height))
```

Image Blending

Images must be identically sized. Uses saturation arithmetic (caps at 255).

Python

```
image1_resized = cv2.resize(image1_rgb, (target_width, target_height))  
blended_image = cv2.add(image1_resized, image2_rgb)
```

Histogram Equalization

Stretches out clumped pixel values across the entire brightness spectrum to improve global contrast.

Python

```
image_gray = cv2.imread('Sukuna.jpeg', cv2.IMREAD_GRAYSCALE)  
equalized_image = cv2.equalizeHist(image_gray)
```